

C++ vector sorting

c++ code for sorting a vector

In C++, you can sort a `vector` easily using the `std::sort` function from the `<algorithm>` header.

Here's a simple example:

```
cpp

#include <iostream>
#include <vector>
#include <algorithm> // for sort()

int main() {
    std::vector<int> v = {5, 2, 9, 1, 5, 6};

    // Sorting in ascending order
    std::sort(v.begin(), v.end());

    std::cout << "Ascending order: ";
    for (int num : v) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

    // Sorting in descending order
    std::sort(v.begin(), v.end(), std::greater<int>());

    std::cout << "Descending order: ";
    for (int num : v) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

◆ Explanation

- `v.begin()` → iterator pointing to the start of the vector.
- `v.end()` → iterator pointing just past the last element.

- `std::sort(v.begin(), v.end())` → sorts in ascending order (default).
- `std::sort(v.begin(), v.end(), std::greater<int>())` → sorts in descending order.

👉 Do you want me to also show you how to implement **sorting a vector manually** (like with Bubble Sort or Merge Sort) instead of using `std::sort`?

explain "std::sort(v.begin(), v.end(), std::greater<int>());"

Good question 👍 Let's break it down:

cpp

```
std::sort(v.begin(), v.end(), std::greater<int>());
```

1. `std::sort(first, last, comp)`

- This is the general form of the `sort` function from `<algorithm>`.
- It sorts the elements in the range `[first, last)` (that means from `first` up to but not including `last`).
- By default, it uses the "less than" (`<`) operator to arrange elements in ascending order.
- But you can give it a **custom comparator** (the `comp` part) to decide the order.

2. `v.begin()` and `v.end()`

- `v.begin()` → iterator pointing to the first element of the vector.
- `v.end()` → iterator pointing just *after* the last element.
- So `sort` will rearrange all elements between these two iterators.

3. `std::greater<int>()`

- `std::greater<int>` is a **function object (functor)** from `<functional>`.
- It defines the operator `()` such that for two elements `a` and `b`:

cpp

```
std::greater<int>()(a, b) // is true if a > b
```

- That means, instead of checking `a < b` (default ascending), the sort will check `a > b`.
 - As a result, it arranges the vector in **descending order**.
-

Example

```
cpp

#include <iostream>
#include <vector>
#include <algorithm>
#include <functional> // for std::greater

int main() {
    std::vector<int> v = {10, 3, 7, 1, 9};

    std::sort(v.begin(), v.end(), std::greater<int>());

    for (int num : v) {
        std::cout << num << " ";
    }
    return 0;
}
```

Output:

```
10 9 7 3 1
```

✅ So, in short:

```
std::sort(v.begin(), v.end(), std::greater<int>());
```

means “Sort the vector from beginning to end in **descending order** by comparing elements using `>` instead of `<`.”

Would you like me to also show how to write your **own comparator function** instead of using `std::greater` ? That way you can customize sorting rules (like sorting by

even first, then odd, etc.).